AZINE

Architecture

Front end, back end, hosting and apps, and why each piece is the way it is

OCTOBER 9, 2026
AZINE.IO · MAIN @ 2483837

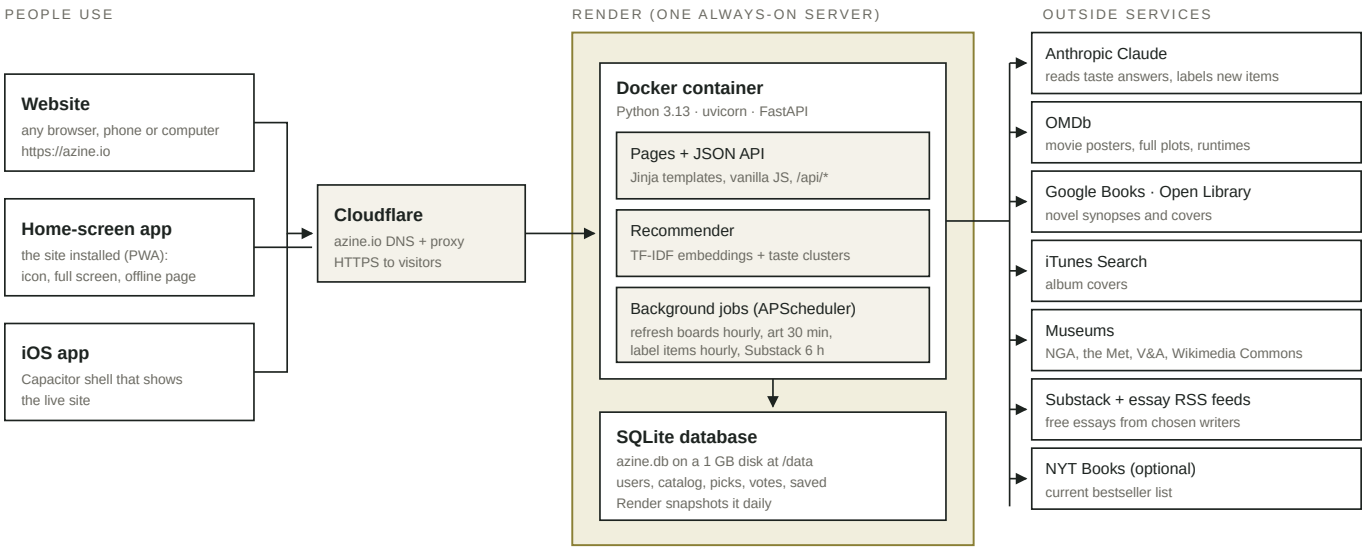
1. What azine is
2. The system at a glance
3. Hosting and deployment
4. Three ways in: website, home-screen app, iOS app
5. Back end: the server
6. Content: catalogs, sources and enrichment
7. Personalization: from a taste answer to a board
8. Front end: the board and the other pages
9. Operations: limits, costs, backups
10. Decision log
11. Keeping this document current

01 What azine is

azine is a personal board of things worth your time: essays, movies, albums and novels, plus works of art from museum collections. It is built for one person at a time from two signals: a short free-text description of what they like and don't, and how they sort each day's picks ("loved it" / "neutral" / "not for me"). It deliberately has no feed, no social layer, no ads, and no AI-written content. Every pick is a real work from a curated catalog, mostly pre-2021 (plus festival films from 2022 on).

> WHY IT'S SHAPED THIS WAY
>
> The product promise is "the best of the internet without the filler." That rules out an engagement feed and generated text, and it argues for a small, finite board that renews on a schedule rather than an infinite scroll. Claude is used only behind the scenes: to understand a person's taste and to label catalog items. It never writes what you read.

02 The system at a glance



Everything runs as one Python server on Render, with its SQLite database on an attached disk. Cloudflare serves the azine.io name; outside services are called only to read taste, label items, and fetch art, covers and synopses.

What happens when someone opens azine.io

1. Cloudflare resolves `azine.io` and passes the request to the Render service over HTTPS (`www.azine.io` and plain `http://` redirect to `https://azine.io`).
2. FastAPI checks the signed session cookie. No session → the landing page (a wall of museum art, then the handwritten note, "Respond here", then sign-up). Signed in but no taste yet → the taste prompt. Otherwise → the board page (HTML from a Jinja template).
3. The board's script calls the JSON API: `/api/recommendations` (the picks), `/api/art`, `/api/saved`, then `/api/board/considering` (the wall's extra images) and `/api/board/vibes` (today's palette, for cover-less cards).
4. If a category's boxes have expired, the recommender refreshes them on the spot; any pick missing a poster, cover or synopsis is looked up once and cached on its catalog row.
5. Votes and saves are small POSTs; in a user's first week a like or dislike immediately rebuilds that category.

---

03 Hosting and deployment

PIECE	SETUP
Host	Render web service, Docker runtime, region Oregon, plan 0.5 CPU / 512 MB (~$7/month), always on
Image	`Dockerfile`: `uv` + Python 3.13 slim; dependencies in their own layer; runs `uvicorn azine.main:app --proxy-headers` on `$PORT`
Disk	1 GB persistent disk mounted at `/data`; `AZINE_DB_PATH=/data/azine.db`. The generated session key lives beside it.
Config	`render.yaml` (Blueprint): disk, health check `/health`, env vars. Secrets (`ANTHROPIC_API_KEY`, `OMDB_API_KEY`, `GOOGLE_BOOKS_API_KEY`, `NYT_API_KEY`) are set in the dashboard, never in git.
Domain	`azine.io` registered on Cloudflare: `A @ → 216.24.57.1`, `CNAME www → azine.onrender.com`. Render issues the certificate; `azine.onrender.com` also still works.
Deploys	Every push to `main` builds and deploys. With a disk attached Render stops the old server first, so there's ~1 minute of downtime. Migrations run on start.
Security	`AZINE_HTTPS_ONLY=1` marks the session cookie Secure; uvicorn trusts the proxy's `X-Forwarded-Proto`.

> WHY ONE SERVER, SQLITE AND A DISK
>
> azine's needs are modest: one database file, an hourly job, and a few hundred requests a day at most. A single always-on process with SQLite on a persistent disk is the simplest setup that does all of that: no separate database server, no job queue, no cluster. The constraints this brings are deliberate: exactly one instance (the SQLite file and the in-process scheduler both assume it), and the disk must exist (the server now refuses to start without it, with a message saying so). The way to grow is a bigger instance; moving to Postgres and a separate worker only becomes necessary if azine ever needs more than one server.

> WHY RENDER (AND NOT THE FREE TIER, FLY.IO, OR VERCEL)
>
> - **Vercel, Netlify, Cloudflare Pages, Heroku** were ruled out: no persistent disk and/or no always-on process, so they'd require Postgres plus an external scheduler.
> - **Fly.io** was set up first (it fits technically), then swapped for **Render** because Render is driven from a web dashboard rather than a command-line tool, and auto-deploys from GitHub.
> - **Render's free tier** sleeps after 15 idle minutes (which would stop the hourly refresh) and can't have a disk.
> - **Docker** keeps it portable: the same image runs on any host that offers a disk at `/data` and HTTPS.

WHY 512 MB IS ENOUGH

Measured peak is ~310 MB across a fresh start, sign-up and every board endpoint. It used to be ~630 MB: the National Gallery of Art fetch held two ~40 MB CSV slices as text (GitHub gzips them, so an "8 MB" byte range is ~40 MB decoded). It now streams the rows and keeps only the three fields it uses. Memory grows with *simultaneous* activity, not total accounts. Upgrade to 1 CPU / 2 GB if the logs ever show it running out.

04 Three ways in: website, home-screen app, iOS app

Website
Server-rendered pages at azine.io; works in any modern browser, phone or desktop. Every app below is a window onto this.

Home-screen app (PWA)
`manifest.webmanifest`, icons, and a service worker (`/sw.js`). "Add to Home Screen" gives an icon, full-screen launch and an offline page. Nothing to publish.

iOS app
`ios-app/`: Capacitor 8 (Swift Package Manager). Loads `https://azine.io`; bundles only the icon, launch screen and offline page. External links open in Safari / Music / YouTube.

Caching and offline

The service worker is **network-first for everything**: pages and the API are never served from a cache (the board is personal and changes daily), and `/static/` files are cached only as an offline fallback. With no connection, a page load shows `static/offline.html`. The stylesheet's address carries its file's modification time (`style.css?v=…`, from `static_url()`), so every edit is a new URL and browsers can never pair a new page with an old stylesheet.

WHY THE APPS WRAP THE WEBSITE

One codebase serves all three, and a deploy updates every app at once, with no App Store release for content or design changes. The trade-off is App Store review guideline 4.2 ("minimum functionality"), which sometimes rejects pure web wrappers: fine for personal use and TestFlight; for the public store, add something native (e.g. push notifications for new picks). Capacitor 8 uses Swift Package Manager, so the Xcode project was generated without CocoaPods.

WHY NETWORK-FIRST CACHING

Originally static files were served cache-first. After the board redesign, a browser paired the new `board.html` with its cached old `style.css` and showed an unstyled page. Versioned stylesheet URLs plus a network-first service worker make that impossible while keeping the offline page.

05 Back end: the server

Code layout (`src/azine/`)

MODULE	JOB
main.py	Builds the FastAPI app: session middleware, static files, routers, manifest/service worker routes, `/health`; on start runs migrations, seeds the catalog, refreshes art, starts the scheduler
db.py · models.py	SQLAlchemy engine/session (SQLite), table definitions, constants (categories, boxes per category, default cadences)
migrations/	Alembic migrations, applied automatically on every start
routers/	HTTP endpoints: pages, auth, quiz/taste, recommendations, votes, board (wall, palette, saved), art, settings

recommender/	embeddings, taste clustering, the engine that fills boxes, refresh service, pick-time enrichment
onboarding.py	A first visit in the background: reads the taste answer before the account exists, then builds the board after sign-up, reporting four stages to the loading screen
taste_interpreter.py · catalog_sampler.py	Claude reads the free-text taste answer; the sampler chooses which slice of the catalog it sees
content_sources/ · data/	Catalog loaders and live sources (Substack, RSS, museums, OMDb, iTunes, books); curated catalogs and vocabularies
labeling.py · catalog_builder.py	Subject/tone labels (hand-edited JSON + Claude for new items); offline tool that builds the big catalogs from source lists
vibes.py · art_refresh.py · scheduler.py · seed.py	The day's palette (colors cover-less cards); museum image rotation; background jobs; catalog upserts on start

HTTP endpoints

ROUTE	PURPOSE
GET / · /quiz · /saved	Pages: the landing page (wall, note, sign-up, loading) or the board; the taste prompt / list quiz (fallback); saved items
GET /about	The owner's letter (public; text in `data/about_letter.txt`)
/robots.txt · /sitemap.xml · /favicon.ico	For search engines: keep out of /api, /quiz, /saved; list the public pages (/, /about)
POST /api/onboarding/start, GET /status	"All done": Claude starts reading the answer before sign-up; progress (stages 1–4) for the loading paintings
POST /api/auth/signup · login · logout, GET /me	Accounts and the session cookie
GET/POST /api/quiz, POST /api/quiz/describe	List quiz; free-text taste answer (first time, or "Refine" updates) read by Claude
GET /api/recommendations	The board's picks by category, refreshing expired boxes
POST /api/recommendations/{id}/vote	"Loved it" / "neutral" / "not for me"; a like or dislike rebuilds the category in a user's first week
GET /api/board/vibes · /considering	Today's palette (+ mood words, no longer shown); the wall's extra images (runner-ups, random catalog, museum gallery)
GET/POST/DELETE /api/saved	Saved items (kept even after a pick rotates out)
GET /api/art · /api/art/gallery	The board's museum images; a larger gallery for the wall and quiz page
GET/POST /api/settings, /connect, /disconnect	Per-category cadence (1–7 days); paywalled outlets you subscribe to
/health · /manifest.webmanifest · /sw.js	Health check for Render; the installable-app files

Accounts and sessions

Passwords are hashed with PBKDF2-SHA256 (260,000 iterations, standard library). The session is Starlette's signed cookie holding only the user id, valid 30 days, `SameSite=Lax`, Secure on the live site. The signing key comes from `AZINE_SECRET_KEY` (generated by Render) or is generated once and stored next to the database.

Data model

TABLE	HOLDS

users	Email, password hash, taste profile (tones/subjects read from the answer), per-category cadence, sign-up date
content_items	The catalog: category, title, creator, year, source, url, image, synopsis, tags, subjects, tones, tier, runtime, publish date
user_interests	Taste signals: liked/disliked items and essay topics, with a weight (Claude's guesses count less)
recommendations	Each box shown: user, item, slot, when generated/expires, the vote and when
saved_items	A copy of each saved pick, so it survives the box rotating out
daily_vibes	Cached palette (+ mood words, unused since the Vibes page was removed) per board
art_images	The current museum images in the board's art slots
essay_platform_connections	Outlets a user subscribes to (New Yorker, Atlantic, Substack)

> WHY MIGRATIONS RUN ON START
>
> The database holds every account and vote, so it must never be deleted to pick up a schema change. Alembic migrations are applied automatically when the server starts, locally and on Render, so a deploy that changes the schema upgrades the live database in place.

Background jobs (APScheduler, inside the server process)

JOB	EVERY	DOES
refresh_recommendations	1 hour	Refreshes any user's category whose cadence has elapsed
refresh_art	30 min	Rotates the board's museum images
label_new_items	1 hour	Claude labels catalog items that have no subjects/tones yet
refresh_substack	6 hours	Pulls new free posts from the chosen Substack newsletters

06 Content: catalogs, sources and enrichment

CATEGORY	CATALOG (≈ ITEMS)	WHERE IT COMES FROM
Movies	1,900	Cross-referenced "greatest films" lists, 1940–2021, plus 2022+ films that screened at Cannes, Sundance, Toronto or Hong Kong. Trailers for some; Library of Congress Screening Room switched off.
Novels	1,900	Cross-referenced "greatest novels" lists, 1810 onward; current NYT bestsellers with a key
Albums	531	Rolling Stone's 500 Greatest Albums, plus BBC Music Magazine's 31 greatest classical albums (`albums_classical_list.py`). Each album is tiered "instrumental" (no words) or "lyrical".
Essays	200 + live	Curated essays from a hand-edited author list, non-paywalled RSS feeds, and chosen Substack newsletters (free posts only)
Art	live	National Gallery of Art open data, the Met (incl. costume), V&A, Wikimedia Commons museum paintings; Library of Congress paused

Enrichment: looked up when picked, cached forever

Most catalog rows start without an image or a full synopsis. When an item is *picked* for someone (or shown on the wall), `recommender/enrich.py` fills it in once and stores it on the row:

- **Movies:** OMDb poster, IMDb link, runtime and the *full* plot (a paragraph, not OMDb's one-liner).

- **Albums:** cover art and the album's Apple Music page from iTunes Search (one lookup gives both).
- **Novels:** synopsis from Google Books (publisher descriptions, several sentences; needs a free key), else Open Library; cover from Open Library (large), else Google's thumbnail. Descriptions are cleaned (HTML, markdown, "From Amazon.com", ALL-CAPS banners), cut at a sentence end past ~900 characters, and non-English text is skipped.

A fuller synopsis replaces a catalog one-liner and survives restarts; each item is looked up at most once per server run.

> WHY AT PICK TIME, NOT IN BULK
>
> Free API quotas are small (OMDb: 1,000 requests/day; Google Books: per-key daily quota, and its keyless quota is shared by everyone and usually exhausted). Enriching ~4,500 items up front would blow through them; enriching only what someone actually sees costs a handful of calls a day. Longer synopses were chosen over one-liners because the board's showcases give the synopsis the main column.

Labels: subjects and tones

Every item is labeled on two axes from one shared vocabulary (`data/labels_vocab.py`): **subjects** (what it's about) and **tones** (how it feels). Curated labels live in `data/catalog_labels.json` (hand-editable, applied on every start); new items are labeled by Claude hourly. Because the vocabulary is shared, a taste read in one category carries into the others: someone drawn to melancholy films gets melancholy albums too.

---

07 Personalization: from a taste answer to a board

1. Reading the taste answer

New users answer one prompt in 20+ words, written under the landing page's handwritten note: what movies, novels, music and reading they want, and what they don't. **"All done" starts Claude reading it immediately, in a background thread** (`onboarding.py`), keyed to the visitor's session cookie; they sign up meanwhile. Sign-up attaches the job to the new account, and as soon as Claude is finished the answer is saved and the board built, a category at a time: stage 1 Claude has read it, 2 movies + novel, 3 albums, 4 essays (last, because the live essay search takes up to ~12 seconds). The loading screen preloads the paintings during sign-up, retries one that fails to load, and goes to the board regardless after 90 seconds. The loading screen fills one painting per stage. If Claude can't read it (no API key) or the server restarted in between, the visitor lands on `/quiz`, which re-reads the answer kept in their browser. `taste_interpreter.py` sends it to Claude (default `claude-sonnet-5-5`) with a sample of the catalog and gets back, as structured output, starting essay topics plus liked and disliked catalog items and the person's overall tones and subjects. These become ordinary `user_interests` rows, so the recommender doesn't care whether a person or the model chose them. "Refine" sends later free text the same way; a new "no more X" overrides an old "I like X", and only affected categories refresh. Without an API key, the pick-from-a-list quiz is used instead.

The catalog is too big to send whole, so `catalog_sampler.py` clusters each category (k-means over the same embeddings) and sends every cluster's most acclaimed titles (identical for everyone, so Claude's prompt cache reuses it), plus anything the answer names outright, plus more from the clusters nearest the answer.

Classical music

Claude is told to put classical albums in the liked items in three cases: the person says they like classical music (it picks the kinds they describe), says they don't really like or listen to music, or says they don't want music with words. In the last case it also sets `instrumental_only` on their taste profile, and the recommender then draws their albums only from the "instrumental" tier (classical without voices, and wordless jazz/ambient from the Rolling Stone list). "Refine" keeps that setting unless a new answer changes it. All 31 classical albums are always in the sample Claude sees, and two (Gould's Goldberg Variations, Kleiber's Beethoven 5 & 7) are options in the list quiz.

2. Embeddings

Each item is a TF-IDF vector blending its labels (genres, subjects, tones; 70% of the weight) with its title and synopsis text (30%). Labels carry most of the signal: measured on the real catalog, 91–94% of same-genre pairs land closer than the nearest 10% of random pairs, versus ~50% for text alone.

3. Taste clusters (after Pinterest's PinnerSage)

- A user's liked items are clustered with Ward hierarchical clustering, cut at a distance threshold, so they get as many clusters as they have distinct tastes.
- Each cluster is represented by its **medoid** (a real item they chose, not an average point).
- Cluster importance = frequency × recency (exponential decay over a 90-day window), scaled by signal weight.
- Each refresh samples up to 3 clusters by importance and gives each a box; the user's overall tones/subjects break ties; dislikes form their own clusters that push candidates away.

> WHY SEVERAL CLUSTERS, AND WHY TF-IDF
>
> A single averaged taste vector blends "dense philosophy" and "light rom-coms" into a bland middle that matches neither; separate clusters let a 10% taste still win a box now and then. TF-IDF (scikit-learn) separates genres well at a catalog of thousands; a neural embedding model would add a multi-gigabyte dependency for little gain at this size. Worth revisiting if the catalog reaches tens of thousands.

4. Boxes, cadence and votes

CATEGORY	BOXES	DEFAULT CADENCE	NOTES
Essays	3	daily	First box is a Substack post when there's an unseen one; bump for watch-list authors and older pieces; penalty for paywalled outlets you haven't connected; unsigned essays held back until you've sorted 20 boxes
Movies	3	weekly	One niche, one well-known, one wildcard from anywhere
Albums	2	every 3 days	
Novels	1	weekly	

Each essay refresh also searches the essay outlets live for the person's topics (12 at a time, at most ~12 seconds overall) and adds what it finds to the catalog. Cadences are adjustable per category (1–7 days) in Settings. For a user's first 7 days, every like or dislike rebuilds that category immediately, so the board converges on a new person's taste while they're still forming it.

A vote is one of three. **Loved it** joins the user's likes (taste clusters, tag profile) and **not for me** their dislikes (clusters that push similar picks away); either way that pick never comes back. **Neutral** only retires the pick: it is never recommended again, but it is left out of every taste signal, so it says nothing about the category or anything like it.

---

08 Front end: the board and the other pages

Pages are server-rendered Jinja templates (`templates/`) with plain JavaScript inside them and one stylesheet (`static/css/style.css`). There is no framework and no build step. Every color, font and size is a variable in the stylesheet's `:root`, documented in place so it can be tuned by hand.

> WHY NO FRAMEWORK
>
> The front end is a handful of pages whose look changes often and is tuned by hand. Plain templates and CSS variables keep every visual decision in one readable file, with nothing to compile and no dependency churn. The board's interactions (scroll-driven showcases, drag-to-sort) are a few hundred lines of vanilla JS.

The board (`board.html`)

1. **The wall**: a fixed, full-screen grid of today's picks' images mixed with runner-ups, random catalog images and museum works (hover for what each is). One scroll, key press or swipe leaves it: today's picks drop away and the rest fade in place. Scrolling up from the very top of the page (as a new gesture, not a fast scroll arriving there) brings the wall back.
2. **Read · Listen · Watch**: one "showcase" per category (novel, essays, albums, movies).
3. **Sort**: every pick in a stack; drag left ("loved it") or right ("not for me"), or use the buttons/arrow keys. The middle button, "neutral", drops the pick out of the stack with no pile.

A palette pulled from the day's images (2 colors per image, up to 8, cached per board) colors the cards that stand in for a missing cover. The "A" in the header opens a menu: saved, refine, settings, logout. Dots on the right show where you are and jump between sections. Tapping an album (its cover, or "Open") floats two glass bubbles beside it, the same as the menu's: "open in Apple Music" (the album's own page) and "open in Spotify" (a search for artist and title). The Saved page links an album to both.

> WHY BOTH, AND WHY SPOTIFY IS A SEARCH
>
> A web page can't see which apps a phone has installed, so both are offered; each link opens its app when it's there and the website when it isn't. Apple Music's album page comes free with the iTunes lookup that already fetches the cover. Finding the exact album on Spotify needs a Spotify developer key and its API; a search link needs neither and lands one tap away.

Showcases (after huyml.co)

Each showcase is as tall as the screen plus one "step" (60% of the screen height) per extra entry. Its frame is `position: sticky`, so it pins while you scroll through. On every scroll frame the script computes *f* (0 = first entry centered, 1.5 = halfway between the 2nd and 3rd) and moves:

- **the covers** along a tilted 3D path (rotate, scale, fade with distance from center);
- **the synopses** on the right so entry *f* sits in the middle, the rest fading;
- **the details** on the left (facts, the title, Open ↗, save) when the nearest entry changes.

- **Essays** have no image: they're their lines of text (label, title, author · date, Open) on borderless translucent cards moving through the same path.
- Movies/novels without an image get a **type card**: the title on one of the day's palette colors.
- "better for your brain" sits in the bottom-left corner in the small label style.
- Clicking any list entry or neighboring cover scrolls it to the center.
- Cover sizes are `--movie-cover-size`, `--album-cover-size`, `--essay-card-size`, `--novel-cover-size`; the step is `STEP` in `board.html`.

> WHY THIS BOARD DESIGN
>
> - The wall keeps a discrete, "cover page" scroll; everything after it scrolls like a normal website because jumping between screens made content disappear as you scrolled.
> - Showcases (modeled on huyml.co) give each multi-entry category one focused entry at a time without hiding the others.
> - The synopsis takes the main column and the title moves into the details: the point is what a work *is*, not just its name.
> - The rolling entry number was replaced by "better for your brain".
> - Proximity scroll-snapping was tried and removed: it pulled the page away from where people stopped (and from the Sort section).
> - Phone layout works but is a known follow-up, to be designed on its own.

Handwriting font

The board's fixed labels are in the owner's handwriting, as a real font: "Azine Hand" (`static/fonts/azine-hand.woff2`, 23 KB). The owner wrote the alphabet (capitals and lowercase, twice each), punctuation and a heart in a drawing app, and the labels with the digits on a second page. Both were exported as *vector* PDFs, where each pen stroke is a filled outline. `scripts/make_handwriting_font.py` groups the strokes into rows and characters, matches them to the expected letters, sets every letter exactly on the baseline (letters that hang below it, like g and the comma, are placed by the row's fitted line), evens the capitals and digits to one height (`EVEN_HEIGHTS`), merges each character's strokes (skia-pathops) and writes the font (fontTools). The two versions of each letter alternate through an OpenType `calt` rule, so double letters don't repeat exactly. Any element with `class="hand"` uses it: section headings, the details' labels, the date (month written out), the menu, the sort labels and buttons, and "Fetching your recs". Titles, synopses and values stay in the regular fonts. Settings: the HANDWRITING block in `style.css`.

> WHY A FONT, NOT IMAGES OF WORDS
>
> A font handles text that changes (the date) and any future label, stays real text (resizable, recolorable by the CSS settings, readable by screen readers), and is one small file. Cut-out word images would match the page exactly but need re-drawing for every wording change. The vector PDFs made the font lossless: the outlines are the pen strokes themselves, not a trace of pixels.

Other pages

auth.html: the landing page in four steps. First a wall of 28 public-domain works from the Met (Hokusai, Vermeer, Bruegel, Kōrin, an Egyptian faience hippo, Ottoman and Mughal manuscript pages…), the same for everyone in the same order (`data/landing_wall.py`; `scripts/make_landing_wall.py` downloads them once and stores them with the site as small WebP files). One scroll, swipe, key or click fades the works away one by one, uncovering the handwritten note full screen (a photo placed on a larger sheet of the same grid paper by `scripts/make_landing_note.py`, so "Respond here" sits on blank paper under the writing) with "All done". On a phone (any portrait screen) the sheet's blank sides are cropped so the writing fills the width, three grid squares below the "A", with the paper tile scaled to keep the grid unbroken. The note's arrow is painted out of the photo and drawn live (an SVG in `auth.html`) from where the pen started down to the "Respond here" box, with the hand-drawn curve stretched to whatever length the screen needs; it draws itself in when the note appears and follows the box as it grows. Then sign-up over the blurred note; then four paintings, same height and each its own width, filling in left to right as the board builds (Monet's *Impression, Sunrise*, Hopper's *High Noon*, Derain's *View of the Thames*, Krøyer's *Summer Evening at Skagen Beach*). The red "A" stays in the corner throughout. Colors and sizes are the LANDING settings in `style.css`. **quiz.html**: the taste prompt (and list-quiz fallback), beside a masonry of museum art. **saved.html**: everything saved, as a grid. Every page includes `_app_head.html` (manifest, icons, service worker) so any of them can be installed. **about.html**: the owner's letter on the note's paper, "ABOUT" in handwriting; public, linked from the landing page and the board's menu.

> WHY A FIXED WALL AND A PUBLIC ABOUT PAGE
>
> The landing page now opens on art rather than text, matching the board people get after signing up. It's fixed (not today's picks) because a visitor has no picks yet, and stored locally so the first screen never waits on a museum API. The About page is the one page of real text a search engine can read; everything else is behind a login. Along with it came the basics search engines look for: a page description and link-preview tags, `robots.txt`, a sitemap and a favicon.

09 Operations: limits, costs, backups

TOPIC	TODAY
Cost	Render web service ~$7/month + small disk; azine.io yearly at Cloudflare; Claude API usage per taste answer/label batch

Memory	~310 MB peak of 512 MB. Watch Render → Metrics; upgrade to 1 CPU / 2 GB if it nears ~450 MB or restarts for memory
API quotas	OMDb 1,000/day (first day after a deploy uses more: one-time full-plot upgrades); Google Books per key
Backups	Render snapshots the disk daily and keeps them ≥7 days (restoring loses later changes). Own copy: `scp -s` over Render SSH (see DEPLOY.md)
Logs	Render → Logs. Startup refuses to run without the `/data` disk and says so
Deploy time	Build a few minutes; a cold start on an empty database takes ~1 minute (seeding + fetching art/Substack)
Scaling path	1) bigger instance; 2) only if more than one server is needed: Postgres + a separate worker for the scheduled jobs

10 Decision log

Newest first. Earlier entries (before Sept 30, 2026) predate this document and are reconstructed from the code and its history.

WHEN	DECISION	WHY (AND WHAT ELSE WAS CONSIDERED)
Oct 9	**A third vote: "neutral"**	Owner's call: some picks are neither loved nor disliked. Neutral means "not this one" without saying anything about the kind of thing, so it only stops that pick coming back. Considered and turned down: a faint like (lots of "meh" would pull the board toward safe middle picks) and a faint dislike (would push away things that were fine). No database change: votes are stored as text.
Oct 9	**Taste answer: 20 words minimum (was 50)**	Owner's call: 50 words asked a lot of a first-time visitor before they'd seen anything. Claude still gets enough to start a board from 20, and "Refine" and the first week's votes fill in the rest.
Oct 8	**Albums open in Apple Music or Spotify**	Owner's request: albums had no link at all. A page can't detect installed apps (no permission exists for it), so tapping an album offers both. Apple Music gets the exact album from the iTunes lookup already used for covers; Spotify gets a search, since an exact lookup would need a Spotify developer key.
Oct 8	**Landing note fits a phone; its arrow is drawn live**	On a phone the whole 30-square sheet was squeezed to the width: the writing was small and ran under "about / log in". Now the writing alone fills the width, below the header. The arrow was part of the photo, so it stopped well short of the box on tall screens; drawn live, it always reaches it. Considered: a separate phone image (the same crop is done in CSS from the one image).
Oct 8	**Handwriting font: letters set exactly on the line**	Owner's report: the letters' bottoms sat on slightly different lines. As written they wandered by up to a sixth of their height and capitals ranged ~600–950 units tall; each letter now sits on the baseline and capitals/digits are evened to one height. The letters keep their shapes; text runs about 7% wider.
Oct 2	**Landing opens on a fixed wall of Met art; About page; SEO basics**	Owner's design: art first, the note second, the same art for every visitor. Lighthouse SEO was 91/100 (no page description) and the only public page's words were in an image; the About page adds readable text, plus description, link previews, robots.txt and sitemap. Also: the Met retired its /v1/search API (410 Gone), which had quietly stopped one of the board's art sources; moved to /v1.1/search.
Oct 1	**Board labels in the owner's handwriting, as a font**	The owner's hand fits azine's "made by a person" point. A font (vs. cut-out word images) handles the date and future labels and stays real text. Built from vector PDFs, so lossless. New wording from the owner's sheet: "Fetching your recs", "Loved it ❤", "Not for me :(", the month written out.
Oct 1	**Vibes page removed from the board**	Owner's call: fun, but not needed. The wall now leads straight into Read. The palette is still computed, because it colors the stand-in cards for picks with no cover; the mood words and the `daily_vibes` table stay (unused) so the page could return without a migration.
Oct 1	**Classical albums: BBC Music Magazine's 31; an "instrumental only" setting**	Rolling Stone's list had no classical. Classical is recommended to people who say they like it, who don't much like music, or who don't want words (they get only wordless albums, enforced in the recommender, not left to Claude). Blurbs are short factual notes (performers, label, year), not the magazine's reviews. Fixed tiers along the way: Trans Europe Express and Another Green World have vocals, and Homogenic / The Diary of Alicia Keys had been tagged classical.

Oct 1	**Essay search runs in parallel with a 12-second budget**	The first live sign-up stuck on the last painting: the live essay search ran 20 outlets × every liked topic one at a time, up to 5 s each, which could take minutes. Now 12 searches run at once and whatever has come back after 12 s is used (8 topics: ~10 s). Also speeds up every user's daily essay refresh.
Oct 1	**Landing page is the handwritten note; Claude reads the answer during sign-up**	The note says why azine exists in the owner's own hand. Reading the answer in the background (instead of after sign-up) hides most of Claude's time behind the sign-up form. Jobs live in memory, fine for one server; /quiz is the fallback if one is lost.
Oct 1	**Four paintings as the loading bar; essays built last**	Discrete stages read better than a spinner. Essays take ~20 s (live search), the rest ~2 s, so essays go last: three paintings fill quickly and the wait sits on the final one. Images: Monet, Derain, Krøyer are public domain; Hopper's *High Noon* (1949) is public domain in the US per Wikimedia Commons (published without a copyright notice) but may be protected elsewhere. Derain's *View of the Thames* (1906) is the image the owner supplied.
Oct 1	**Domain azine.io on Cloudflare**	azine.com is registered (since 1998) and parked for sale on Sedo; azine.io was free. Cloudflare records must be "DNS only" while Render verifies and issues the certificate.
Oct 1	**Refuse to start without the /data disk**	A service created without its disk crashed with SQLite's unhelpful "unable to open database file"; running without the disk would silently lose every account on each deploy.
Oct 1	**Versioned stylesheet URL + network-first service worker**	A cached old stylesheet with the new board showed an unstyled page. Considered: cache-busting by hand. Chosen: automatic, from the file's modification time.
Oct 1	**Movies use OMDb's full plot**	Multi-sentence synopses read better in the showcase's main column; one-time upgrade for movies already picked.
Oct 1	**Novel synopses + covers from Google Books / Open Library**	90% of novels had no synopsis or cover. Google Books has the fuller descriptions but needs a key (keyless quota is shared and exhausted); Open Library is the fallback and the cover source. Claude-written synopses were considered and not used (no AI-written content).
Oct 1	**Showcases: synopsis right, title left; essays as translucent text cards; no borders; "better for your brain" replaces the number**	Owner's design direction after seeing the first version.
Oct 1	**Wall stays discrete; the rest scrolls normally; showcases after huyml.co**	Discrete screens made content vanish on scroll. Unused wall images now fade in place instead of floating up. Scroll-snapping tried and removed.
Oct 1	**Stream the NGA art CSVs**	The fetch used ~400 MB to pick 2 images (gzip ranges decode ~5× larger). Server peak 630 → 310 MB, fitting Render's 512 MB size.
Oct 1	**Host on Render (replacing Fly.io)**	Dashboard-driven, auto-deploys from GitHub. Free tier ruled out; Vercel/Netlify/Heroku ruled out (no disk / no always-on process).
Oct 1	**App icon is the "A"; launch screen keeps the wordmark**	The "A" became the logo in the header; a square mark reads better as an icon.

Sept 30	**iOS app as a Capacitor wrapper; installable web app**	One codebase, updates without App Store releases. Native rewrite considered: far more work for the same content. Risk: App Store guideline 4.2.
Sept 30	**Docker image + configurable database path**	Portable across hosts; the database lives on a disk that survives deploys.
Sept 30	**Saved items and daily vibes (migration 0003)**	Saved copies outlive rotating boxes; vibes are computed once per board instead of on every load.
Sept 29	**Shared subject/tone labels + taste profile (migration 0002)**	Lets a taste read in one category carry into the others; Claude's guesses weighted below explicit choices.
Sept 28	**SQLite + Alembic migrations applied on start (baseline 0001)**	Never delete the database to change the schema; migrations upgrade it in place everywhere.
Earlier	**Free-text taste prompt read by Claude, list quiz as fallback**	One natural answer replaces a long quiz, and captures dislikes too. Claude only chooses from the real catalog.
Earlier	**PinnerSage-style taste clusters over TF-IDF embeddings**	Preserves distinct tastes instead of averaging them; TF-IDF separates genres well without a multi-GB model.
Earlier	**Enrich items when picked, not in bulk**	Stays inside free API quotas.
Earlier	**Server-rendered pages, vanilla JS, CSS variables**	Hand-tunable look, nothing to build.

11 Keeping this document current

The sources live in `docs/architecture/` (`architecture.html`, `summary.html`, `system.svg`, `print.css`); the PDFs are `docs/azine-architecture.pdf` and `docs/azine-summary.pdf`. After any significant change (hosting, data model, a new feature or flow, a major design change, a new outside service) update the affected sections, add a row to the decision log, rebuild with `node docs/architecture/build.js`, and commit the PDFs alongside the change. `CLAUDE.md` asks Claude to do this and to share the updated PDFs.